

Integrantes: Daniel Jiménez Pluma y Santiago Rivera Tetetla

Carrera: Ingeniería en Tecnologías de la Información e Innovación Digital

Matrícula: 2431124958

Nombre de la asignatura: Aplicaciones Web

Nombre de la profesora: Diana Xóchitl Ruano Vargas

Actividad: Documentación de despliegue web

Fecha: 07/04/2026



DOCUMENTACIÓN FINAL DEL PROYECTO

Sistema Web "El Chinaloense Oficial"

1. INTRODUCCIÓN

El presente proyecto tiene como propósito el desarrollo de una aplicación web para el restaurante **El Chinaloense Oficial**, con el objetivo principal de digitalizar y optimizar el proceso de pedidos mediante el uso de tecnologías modernas.

Antes de la implementación del sistema, el restaurante gestionaba sus pedidos de forma manual, utilizando principalmente llamadas telefónicas y mensajes de WhatsApp. Este método, aunque funcional en un inicio, generaba múltiples problemáticas conforme aumentaba la demanda del negocio.

Entre los principales problemas detectados se encuentran:

- **Errores en la toma de pedidos:** debido a la comunicación verbal o mensajes mal interpretados.
- **Saturación en horas pico:** dificultad para atender múltiples clientes al mismo tiempo.
- **Falta de organización:** los pedidos no seguían un orden estructurado.
- **Ausencia de registro:** no existía un historial de pedidos ni control de ventas.

Ante esta situación, se desarrolló un sistema web que permite:

- Mostrar un **menú digital interactivo**
- Permitir al usuario **seleccionar productos**
- Gestionar un **carrito de compras**
- Enviar pedidos directamente al sistema del restaurante

Este sistema mejora tanto la experiencia del cliente como la eficiencia operativa del negocio, ya que automatiza procesos y reduce errores humanos.

2. ARQUITECTURA DEL SISTEMA

El sistema fue diseñado bajo una arquitectura **cliente-servidor**, lo cual significa que se divide en dos partes principales:

- **Frontend (cliente):** interfaz con la que interactúa el usuario
- **Backend (servidor):** encargado de procesar datos y lógica del sistema

¿Por qué se eligió esta arquitectura?

Se eligió esta arquitectura porque permite organizar el sistema de manera estructurada y eficiente. Cada parte tiene una responsabilidad específica, lo que facilita el desarrollo y mantenimiento.

Ventajas:

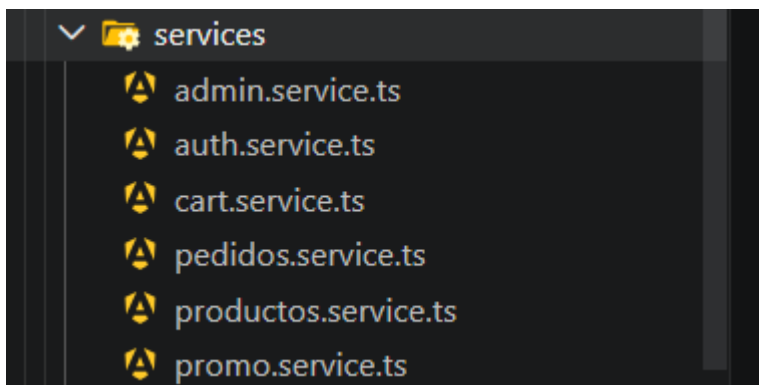
- Permite modificar el frontend sin afectar el backend
- Mejora la seguridad al no exponer directamente la base de datos
- Facilita escalar el sistema en el futuro
- Permite integrar otras tecnologías si se requiere

Funcionamiento detallado del sistema

1. El usuario abre la página web y navega en el menú (Angular)
2. Cuando realiza una acción (ej. iniciar sesión o hacer pedido), Angular envía una solicitud HTTP
3. Esta solicitud llega al backend (Flask)
4. Flask interpreta la solicitud y ejecuta la lógica correspondiente
5. Si es necesario, consulta o guarda información en MongoDB
6. Flask envía una respuesta al frontend
7. Angular recibe la respuesta y actualiza la interfaz en tiempo real

Este flujo permite que el sistema sea dinámico e interactivo.

Servicios Utilizados:



Admin.service.ts

```
el-chinaloense-oficial > src > app > services > admin.service.ts > AdminService
You, la semana pasada | 1 author (You)
1 import { Injectable } from '@angular/core';
2 import { PedidosService } from '../pedidos.service';
3 import { ProductosService } from '../productos.service';
4
5 @Injectable({
6   providedIn: 'root'
7 })
8 export class AdminService {
9
10   constructor(
11     private pedidosService: PedidosService,
12     private productosService: ProductosService
13   ) {}
14
15   // 🔥 FUNCIÓN PARA NORMALIZAR FECHA
16   private esMismoDia(fechaPedido: string) {
17
18     if (!fechaPedido) return false;
19
20     const hoy = new Date();
21     const fecha = new Date(fechaPedido);
22
23     return (
24       fecha.getDate() === hoy.getDate() &&
25       fecha.getMonth() === hoy.getMonth() &&
26       fecha.getFullYear() === hoy.getFullYear()
27     );
28   }
29 }
```

```

30
31 // 🍕 PEDIDOS DE HOY
32 getPedidosHoy(callback: (cantidad: number) => void) {
33
34     this.pedidosService.getPedidos().subscribe(pedidos => {
35         You, hace 3 semanas * 1
36         const pedidosHoy = pedidos.filter((p:any) =>
37             this.esMismoDia(p.fecha)
38         );
39
40         callback(pedidosHoy.length);
41
42     });
43
44 }
45
46 // 🍕 VENTAS DE HOY (SOLO ENTREGADOS)
47 getVentasHoy(callback: (total: number) => void) {
48
49     this.pedidosService.getPedidos().subscribe(pedidos => {
50
51         const pedidosHoy = pedidos.filter((p:any) =>
52             this.esMismoDia(p.fecha) && p.estado === 'entregado'
53         );
54
55         const total = pedidosHoy.reduce((sum:number, p:any) => {
56             return sum + (p.total || 0);
57         }, 0);
58
59         callback(total);
60
61     });
62
63 }

```

```

54
55 // 🍕 VENTAS DE LA SEMANA
56 getVentasSemana(callback: (total:number)=>void) {
57
58     this.pedidosService.getPedidos().subscribe(pedidos => {
59
60         const hoy = new Date();
61         const hace7 = new Date();
62         hace7.setDate(hoy.getDate() - 7);
63
64         const pedidosSemana = pedidos.filter((p:any) => {
65
66             if(!p.fecha || p.estado !== 'entregado') return false;
67
68             const fechaPedido = new Date(p.fecha);
69
70             return fechaPedido >= hace7 && fechaPedido <= hoy;
71
72         });
73
74         const total = pedidosSemana.reduce((sum:number, p:any) => {
75             return sum + (p.total || 0);
76         }, 0);
77
78         callback(total);
79
80     });
81
82 }
83
84
85
86
87
88
89
90
91
92

```

```

94 // 🔥 VENTAS DEL MES (NUEVO BIEN HECHO)
95 getVentasMes(callback: (total:number)=>void) {
96
97     this.pedidosService.getPedidos().subscribe(pedidos => {
98
99         const hoy = new Date();
100         const mesActual = hoy.getMonth();
101         const añoActual = hoy.getFullYear();
102
103         const pedidosMes = pedidos.filter((p:any) => {
104
105             if(!p.fecha || p.estado !== 'entregado') return false;
106
107             const fechaPedido = new Date(p.fecha);
108
109             return (
110                 fechaPedido.getMonth() === mesActual &&
111                 fechaPedido.getFullYear() === añoActual
112             );
113         });
114
115         const total = pedidosMes.reduce((sum:number, p:any) => {
116             return sum + (p.total || 0);
117         }, 0);
118
119         callback(total);
120
121     });
122 }
123
124

```

```

125
126 // 🔥 TOTAL DE PLATILLOS
127 getPlatillos(callback: (cantidad:number)=>void) {
128
129     this.productosService.getProductos().subscribe(data => {
130         callback(data.length);
131     });
132 }
133
134
135 // 🔥 PEDIDOS PENDIENTES
136 getPendientes(callback: (cantidad:number) => void) {
137
138     this.pedidosService.getPedidos().subscribe(pedidos => {
139
140         const pendientes = pedidos.filter((p:any) =>
141             | p.estado === 'pendiente'
142         );
143
144         callback(pendientes.length);
145
146     });
147 }
148
149
150 }

```

Auth.service.ts

```
el-chinaloense-oficial > src > app > services > 🚩 auth.service.ts > 🚩 AuthService
You, la semana pasada | 1 author (You)
1 import { Injectable } from '@angular/core';
2 import { HttpClient } from '@angular/common/http';
3
4 You, la semana pasada | 1 author (You)
5 @Injectable({
6   providedIn: 'root'
7 })
8 export class AuthService {
9
10   private API = 'https://elchinaloense-backend.onrender.com';
11
12   constructor(private http: HttpClient) {}
13
14   login(email: string, password: string, callback: (rol:any)=>void){
15
16     this.http.post(`${this.API}/login`, {
17       correo: email,
18       password: password
19     }).subscribe((res:any) => {
20
21       const usuario = res.usuario;
22
23       let rol = 'cliente';
24
25       if(usuario.correo === 'admin@admin.com'){
26         rol = 'admin';
27       }
28
29       const usuarioFinal = {
30         nombre: usuario.nombre,
31         email: usuario.correo,
32         rol: rol
33
34         localStorage.setItem('usuario', JSON.stringify(usuarioFinal));
35
36         callback(rol);
37
38       }, () => {
39
40         alert('Correo o contraseña incorrectos');
41         callback(null);
42
43       });
44
45     }
46
47     registro(data: any){
48       return this.http.post(`${this.API}/registro`, data);
49     }
50
51     getUsuario(){
52       return JSON.parse(localStorage.getItem('usuario') || 'null');
53     }
54
55     esAdmin(){
56       const usuario = this.getUsuario();
57       return usuario?.rol === 'admin';
58     }
59
60     estaLogueado(){
61       return this.getUsuario() !== null;
62     }
63
64     logout(){
65       localStorage.removeItem('usuario');
```

Cart.service.ts

```
el-chinaiodense-oficial / src / app / services / cart.service.ts > % CartService
You, hace 3 semanas | 1 author (You)
1 import { Injectable } from '@angular/core';
2 import { BehaviorSubject } from 'rxjs';
3
4 You, hace 3 semanas | 1 author (You)
5 @Injectable({
6   providedIn: 'root'
7 })
8 export class CartService {
9
10   items: any[] = [];
11
12   private totalItemsSubject = new BehaviorSubject<number>(0);
13   totalItems$ = this.totalItemsSubject.asObservable();
14
15   constructor() {
16
17     const savedCart = localStorage.getItem('cart');
18
19     if (savedCart) {
20       this.items = JSON.parse(savedCart);
21       this.totalItemsSubject.next(this.items.length);
22     }
23   }
24
25   addToCart(product: any) {
26
27     this.items.push(product);
28
29     localStorage.setItem('cart', JSON.stringify(this.items));
30
31     this.totalItemsSubject.next(this.items.length);
32
33   }
34
35   getItems() {
36
37     return this.items;
38   }
39
40   removeItem(index: number) {
41
42     this.items.splice(index, 1);
43
44     localStorage.setItem('cart', JSON.stringify(this.items));
45
46     this.totalItemsSubject.next(this.items.length);
47   }
48
49   getTotal() {
50
51     return this.items.reduce((total, item) => total + (item.precioFinal || item.precio), 0);
52   }
53
54   clearCart() {
55
56     this.items = [];
57
58     localStorage.removeItem('cart');
59
60     this.totalItemsSubject.next(0);
61   }
62
63 }
64
65
66
67 }
```


Pedidos.service.ts

```
el-chinaloense-oficial > src > app > services > 🚩 pedidos.service.ts > 🐞 PedidosService
You, la semana pasada | 1 author (You)
1 import { Injectable } from '@angular/core';
2 import { HttpClient } from '@angular/common/http';
3 import { Observable } from 'rxjs';
4
5 You, la semana pasada | 1 author (You)
6 @Injectable({
7   providedIn: 'root'
8 })
9 export class PedidosService {
10
11   private API = 'https://elchinaloense-backend.onrender.com';
12
13   constructor(private http: HttpClient) {}
14
15   // 🔥 OBTENER PEDIDOS
16   getPedidos(): Observable<any[]> {
17     return this.http.get<any[]>(`${this.API}/pedidos`);
18   }
19
20   // 🔥 CREAR PEDIDO
21   guardarPedido(pedido: any): Observable<any> {
22     return this.http.post(`${this.API}/pedidos`, pedido);
23   }
24
25   // 🔥 ACTUALIZAR ESTADO
26   actualizarEstado(id: string, estado: string): Observable<any> {
27     return this.http.put(`${this.API}/pedidos/${id}`, { estado });
28   }
29 }
```

Productos.service.ts

```
el-chinaloense-oficial > src > app > services > 🚩 productos.service.ts > 🐞 ProductosService
You, la semana pasada | 1 author (You)
1 import { Injectable } from '@angular/core'
2 import { HttpClient } from '@angular/common/http'
3 import { Observable, map } from 'rxjs'
4 import { Producto } from '../models/producto'
5
6 You, la semana pasada | 1 author (You)
7 @Injectable({
8   providedIn: 'root'
9 })
10 export class ProductosService {
11
12   private apiUrl = 'https://elchinaloense-backend.onrender.com/platillos'
13
14   constructor(private http: HttpClient) {}
15
16   getProductos(): Observable<Producto[]> {
17     return this.http.get<any[]>(this.apiUrl).pipe(
18       map(data => data.map(p => ({
19         ...p,
20         ingredientes: p.ingredientes || [],
21         extras: p.extras || [],
22         imagen: p.imagen || '',
23         nombre: p.nombre || '',
24         descripcion: p.descripcion || '',
25         categoria: p.categoria || '',
26         precio: p.precio || 0
27       })))
28   )
29
30   agregarProducto(producto: Producto): Observable<any> {
31     return this.http.post(this.apiUrl, producto)
32   }
33
34   actualizarProducto(id: string, producto: Producto): Observable<any> {
35     return this.http.put(`${this.apiUrl}/${id}`, producto)
36   }
37
38   eliminarProducto(id: string): Observable<any> {
39     return this.http.delete(`${this.apiUrl}/${id}`)
40   }
41
42 }
```

Promo.service.ts

```
el-chinaloense-oficial > src > app > services > 📁 promo.service.ts > 🐞 PromoService
You, la semana pasada | 1 author (You)
1 import { Injectable } from '@angular/core';
2 import { Promo } from '../models/promo';
3
4 @Injectable({
5   providedIn: 'root'
6 })
7 export class PromoService {
8
9   private STORAGE_KEY = 'promoDelDia'
10
11   promo: Promo = {
12     titulo: 'Promoción del día',
13     descripcion: 'Ordena cualquier aguachile y recibe una bebida gratis',
14     imagen: 'assets/promo.jpg'
15   }
16
17   constructor(){
18     this.cargarPromo()
19   }
20
21
22   private cargarPromo(){
23
24     try{
25
26       const data = localStorage.getItem(this.STORAGE_KEY)
27
28       if(data){
29         this.promo = JSON.parse(data)
30       }else{
31         this.guardarPromo()
32       }
33     }catch(error){
34       console.error('Error cargando promo:', error)
35       this.guardarPromo()
36     }
37   }
38
39
40
41
42   private guardarPromo(){
43     localStorage.setItem(this.STORAGE_KEY, JSON.stringify(this.promo))
44   }
45
46   You, la semana pasada • .. ...
47   getPromo(){
48     return this.promo
49   }
50
51
52   actualizarPromo(nuevaPromo: Promo){
53     this.promo = nuevaPromo
54     this.guardarPromo()
55   }
56
57 }
```

Modelos utilizados:

Pedido.ts

```
el-chinaloense-oficial > src > app > models > TS ped
You, hace 3 semanas | 1 author (You)
1  export interface Pedido {
2
3    id:number
4
5    productos:any[]
6
7    total:number
8
9    estado:string
10
11   fecha:Date
12   ⚡
13 } You, hace 3 semanas • 1
```

Producto.ts

```
el-chinaloense-oficial > src > app > models > TS producto.ts > Producto
You, la semana pasada | 1 author (You)
1  export interface Extra {
2    nombre: string
3    precio: number
4  }
5
6  You, la semana pasada | 1 author (You)
7  export interface Producto {
8    _id?: string // 🔥 Mongo ID
9
10   nombre: string
11   descripcion: string
12   precio: number
13   imagen: string
14   categoria: string
15
16   ingredientes: string[]
17   extras: Extra[]
18
19   popular?: boolean
20 } You, hace 3 semanas • 1 ...
```

Promo.ts

```
el-chinaloense-oficial > src > app > models > ts promo.ts >  Promo  
You, hace 3 semanas | 1 author (You)  
1 export interface Promo{  
2  
3   titulo:string  
4  
5   descripcion:string  
6  
7   imagen:string  
8  
9 }
```

App.py

```
You, la semana pasada | 1 author (You)  
1 from flask import Flask, request, jsonify  
2 from flask_cors import CORS  
3 from pymongo import MongoClient  
4 from bson import ObjectId  
5 import os  
6  
7 app = Flask(__name__)  
8 CORS(app)  
9  
10 client = MongoClient(os.environ.get("MONGO_URI"))  
11  
12 db = client["chinaloenseDB"]  
13 collection = db["platillos"]  
14 collection_pedidos = db["pedidos"]  
15 collection_usuarios = db["usuarios"]  
16  
17 @app.route("/")  
18 def home():    You, la semana pasada * .  
19     return "Backend funcionando"  
20  
21  
22 # =====  
23 # PLATILLOS  
24 # =====  
25  
26 @app.route("/platillos", methods=["GET"])  
27 def obtener_platillos():  
28     try:  
29         lista = []  
30         for item in collection.find():  
31             item["_id"] = str(item["_id"])  
32             lista.append(item)  
33         return jsonify(lista), 200  
34     except Exception as e:  
35         return jsonify({"error": str(e)}), 500
```

```

38 @app.route("/platillos", methods=["POST"])
39 ∨ def agregar_platillo():
40 ∨     try:
41         data = request.get_json()
42
43         if not data:
44             return jsonify({"error": "Datos vacíos"}), 400
45
46         data.pop("_id", None)
47         data.pop("id", None)
48
49         result = collection.insert_one(data)
50
51         return jsonify({
52             "mensaje": "Platillo agregado",
53             "id": str(result.inserted_id)
54         }), 201
55
56 ∨     except Exception as e:
57         return jsonify({"error": str(e)}), 500
58
59
60 @app.route("/platillos/<id>", methods=["PUT"])
61 ∨ def actualizar_platillo(id):
62 ∨     try:
63         data = request.get_json()
64
65         if not data:
66             return jsonify({"error": "Datos vacíos"}), 400
67
68         data.pop("_id", None)
69         data.pop("id", None)
70
71         collection.update_one(
72             {"_id": ObjectId(id)},
73             {"$set": data}

```

```

75         return jsonify({"mensaje": "Platillo actualizado"}), 200
76
77     except Exception as e:
78         return jsonify({"error": str(e)}), 500
79
80
81
82 @app.route("/platillos/<id>", methods=["DELETE"])
83 def eliminar_platillo(id):
84     try:
85         collection.delete_one({"_id": ObjectId(id)})
86         return jsonify({"mensaje": "Platillo eliminado"}), 200
87     except Exception as e:
88         return jsonify({"error": str(e)}), 400
89
90
91 # =====
92 # PEDIDOS
93 # =====
94
95 @app.route("/pedidos", methods=["POST"])
96 def crear_pedido():
97     try:
98         data = request.get_json()
99
100         if not data:
101             return jsonify({"error": "Datos vacíos"}), 400
102
103         result = collection_pedidos.insert_one(data)
104
105         return jsonify({
106             "mensaje": "Pedido guardado",
107             "id": str(result.inserted_id)
108         }), 201

```

```

110     except Exception as e:
111         print("ERROR PEDIDO:", e)
112         return jsonify({"error": str(e)}), 500
113
114
115 @app.route("/pedidos", methods=["GET"])
116 def obtener_pedidos():
117     try:
118         lista = []
119
120         for item in collection_pedidos.find():
121             item["_id"] = str(item["_id"])
122             lista.append(item)
123
124         return jsonify(lista), 200
125
126     except Exception as e:
127         print("ERROR GET PEDIDOS:", e)
128         return jsonify({"error": str(e)}), 500
129
130
131 @app.route("/pedidos/<id>", methods=["PUT"])
132 def actualizar_estado_pedido(id):
133     try:
134         data = request.get_json()
135
136         if not data or "estado" not in data:
137             return jsonify({"error": "Estado requerido"}), 400
138
139         resultado = collection_pedidos.update_one(
140             {"_id": ObjectId(id)},
141             {"$set": {"estado": data["estado"]}}
142         )
143

```

```

144         if resultado.matched_count == 0:
145             return jsonify({"error": "Pedido no encontrado"}), 404
146
147         return jsonify({"mensaje": "Estado actualizado"}), 200
148
149     except Exception as e:
150         print("ERROR PUT PEDIDO:", e)
151         return jsonify({"error": str(e)}), 500
152
153
154     # =====
155     # USUARIOS (LOGIN / REGISTRO)
156     # =====
157
158     @app.route("/registro", methods=["POST"])
159     def registro():
160         try:
161             data = request.get_json()
162
163             if not data:
164                 return jsonify({"error": "Datos vacíos"}), 400
165
166             usuario_existente = collection_usuarios.find_one({
167                 "correo": data.get("correo")
168             })
169
170             if usuario_existente:
171                 return jsonify({"error": "El usuario ya existe"}), 400
172
173             collection_usuarios.insert_one(data)
174
175             return jsonify({"mensaje": "Usuario registrado"}), 201
176
177         except Exception as e:
178             print("ERROR REGISTRO:", e)
179             return jsonify({"error": str(e)}), 500
180
181
182     @app.route("/login", methods=["POST"])
183     def login():
184         try:
185             data = request.get_json()
186
187             usuario = collection_usuarios.find_one({
188                 "correo": data.get("correo"),
189                 "password": data.get("password")
190             })
191
192             if not usuario:
193                 return jsonify({"error": "Credenciales incorrectas"}), 401
194
195             usuario["_id"] = str(usuario["_id"])
196
197             return jsonify({
198                 "mensaje": "Login exitoso",
199                 "usuario": usuario
200             }), 200
201
202         except Exception as e:
203             print("ERROR LOGIN:", e)
204             return jsonify({"error": str(e)}), 500
205
206
207     if __name__ == "__main__":
208         app.run()

```


3. HERRAMIENTAS UTILIZADAS

En el desarrollo del sistema se utilizaron diversas herramientas tecnológicas, cada una con un propósito específico dentro del proyecto. La correcta elección de estas herramientas permitió construir un sistema funcional, escalable y eficiente.

Visual Studio Code

Visual Studio Code fue utilizado como el entorno de desarrollo principal, ya que permite trabajar de manera integral tanto el frontend como el backend en un mismo espacio.

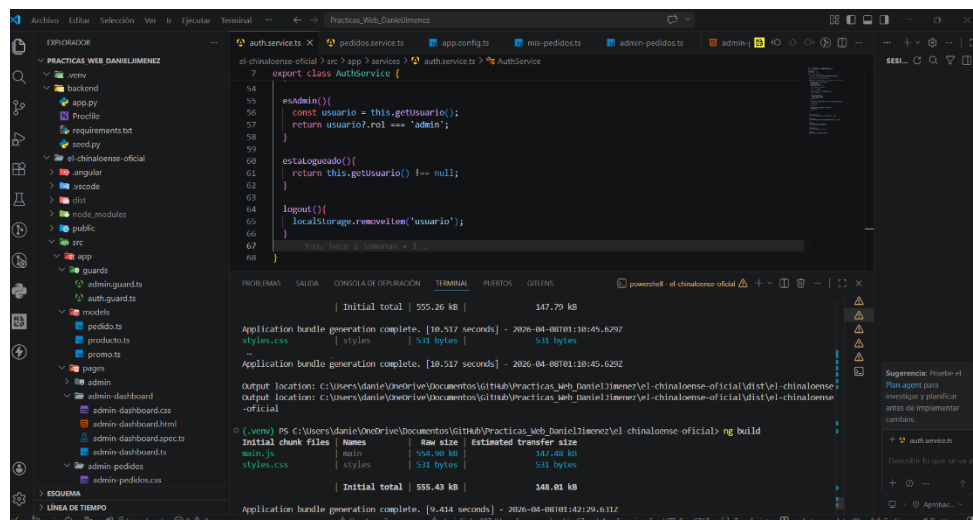
¿Para qué se utilizó?

- Escribir y editar el código fuente del proyecto
- Organizar la estructura de carpetas y archivos
- Ejecutar comandos mediante su terminal integrada
- Instalar extensiones que facilitan el desarrollo

¿Por qué es importante?

Esta herramienta permitió mejorar la productividad del desarrollo gracias a funciones como:

- Autocompletado inteligente de código
- Detección de errores en tiempo real
- Integración con herramientas como Git
- Uso de Thunder Client para pruebas de API



Angular CLI

Angular CLI es una herramienta esencial para el desarrollo del frontend.

¿Para qué se utilizó?

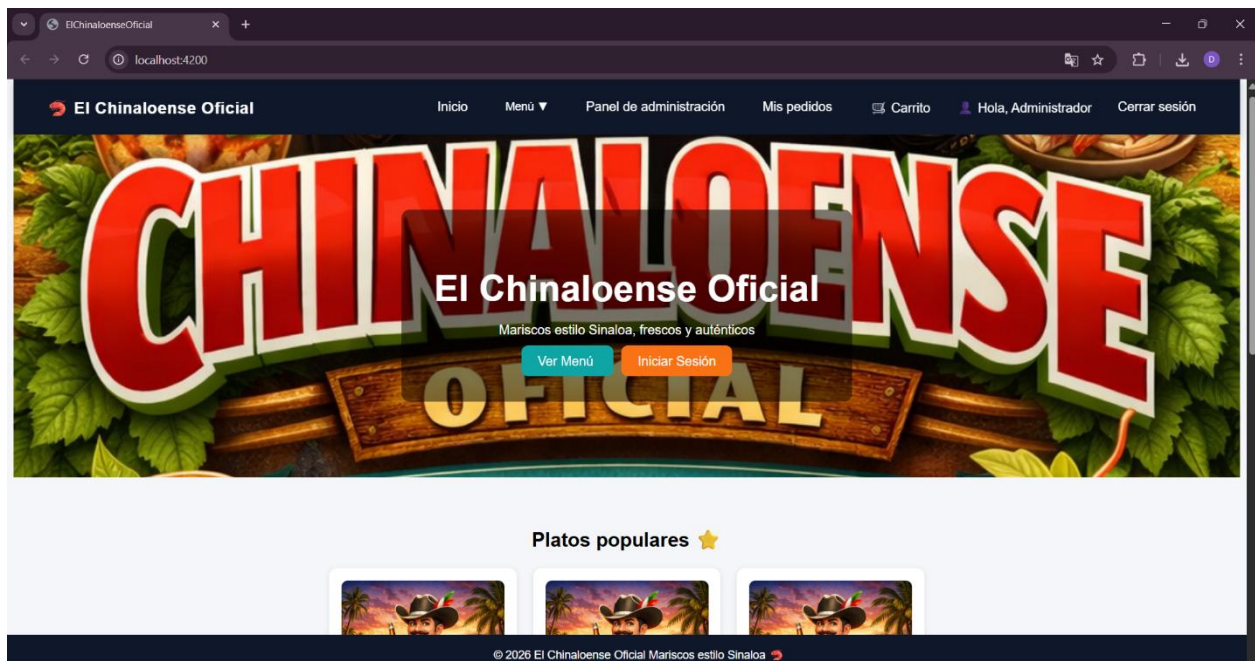
- Crear la estructura base del proyecto
- Generar componentes automáticamente
- Ejecutar el servidor local para pruebas
- Compilar el proyecto para producción

¿Por qué es importante?

Permite automatizar tareas complejas, evitando errores manuales y acelerando el desarrollo.

Por ejemplo:

- Con un solo comando se pueden crear componentes completos
- Permite levantar el proyecto en segundos para visualizar cambios



Python

Python fue utilizado como el lenguaje principal del backend.

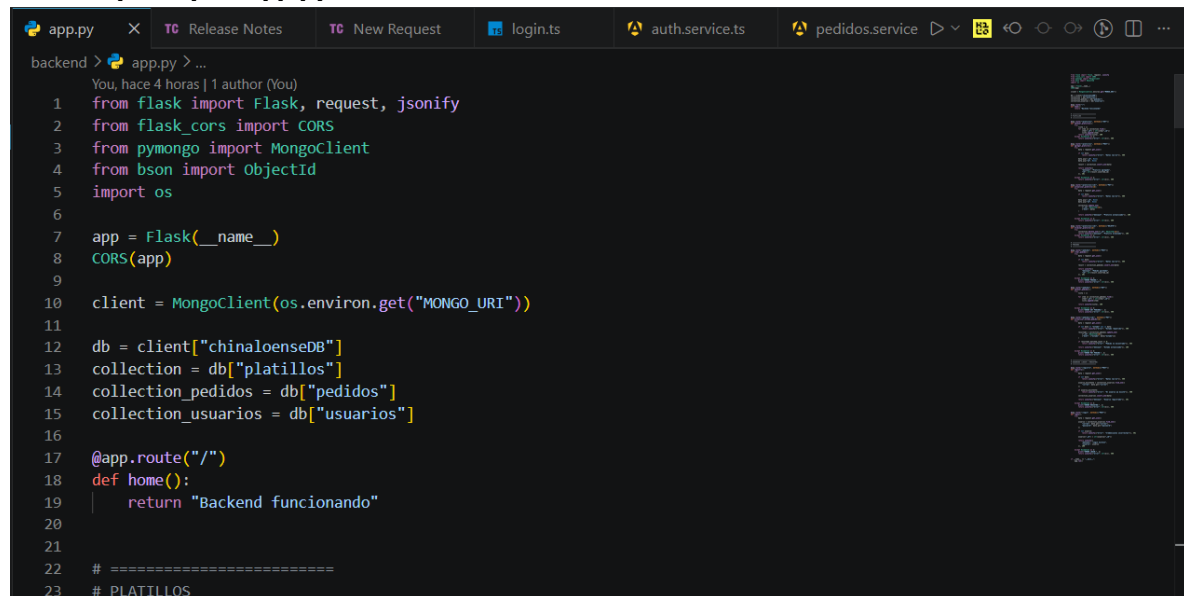
¿Para qué se utilizó?

- Procesar la lógica del sistema
- Manipular datos enviados desde el frontend
- Conectar con la base de datos

¿Por qué es importante?

Python es un lenguaje sencillo pero muy potente, lo que permitió desarrollar el backend de forma clara, rápida y eficiente.

Archivo principal (app.py)



```
app.py X TC Release Notes TC New Request login.ts auth.service.ts pedidos.service
backend > app.py > ...
You, hace 4 horas | 1 author (You)
1 from flask import Flask, request, jsonify
2 from flask_cors import CORS
3 from pymongo import MongoClient
4 from bson import ObjectId
5 import os
6
7 app = Flask(__name__)
8 CORS(app)
9
10 client = MongoClient(os.environ.get("MONGO_URI"))
11
12 db = client["chinaloenseDB"]
13 collection = db["platillos"]
14 collection_pedidos = db["pedidos"]
15 collection_usuarios = db["usuarios"]
16
17 @app.route("/")
18 def home():
19     return "Backend funcionando"
20
21
22 # =====
23 # PLATILLOS
```

Flask

Flask es el framework que permitió construir la API del sistema.

¿Para qué se utilizó?

- Crear rutas (endpoints)
- Recibir solicitudes HTTP (GET, POST, PUT)

- Procesar información
- Enviar respuestas al frontend

¿Cómo funciona en el sistema?

Cada acción del usuario (login, pedido, etc.) está conectada a una ruta en Flask. Cuando Angular envía una petición, Flask la recibe, ejecuta la lógica y devuelve una respuesta.

Render

Render es la plataforma utilizada para desplegar el backend.

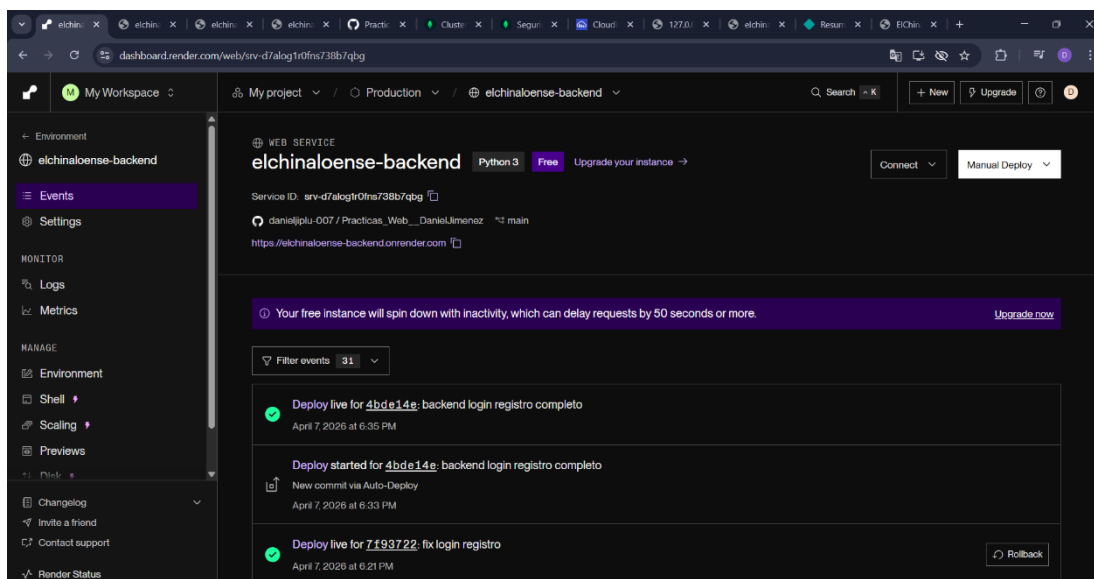
¿Para qué se utilizó?

- Publicar el servidor en internet
- Mantener la API activa 24/7
- Permitir que el frontend pueda conectarse desde cualquier lugar

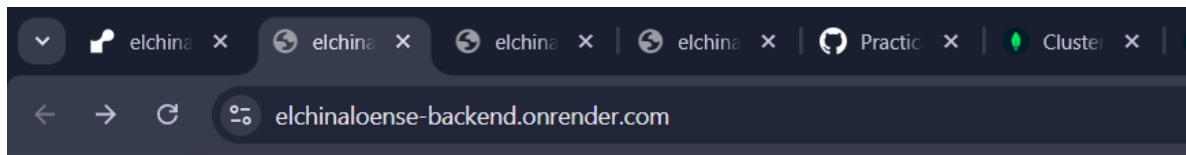
¿Por qué es importante?

Antes del deploy, el backend solo funcionaba en local. Con Render, el sistema se vuelve accesible globalmente.

• Dashboard de Render:



- **Backend en ejecución:**



Backend funcionando

Netlify

Netlify se utilizó para desplegar el frontend.

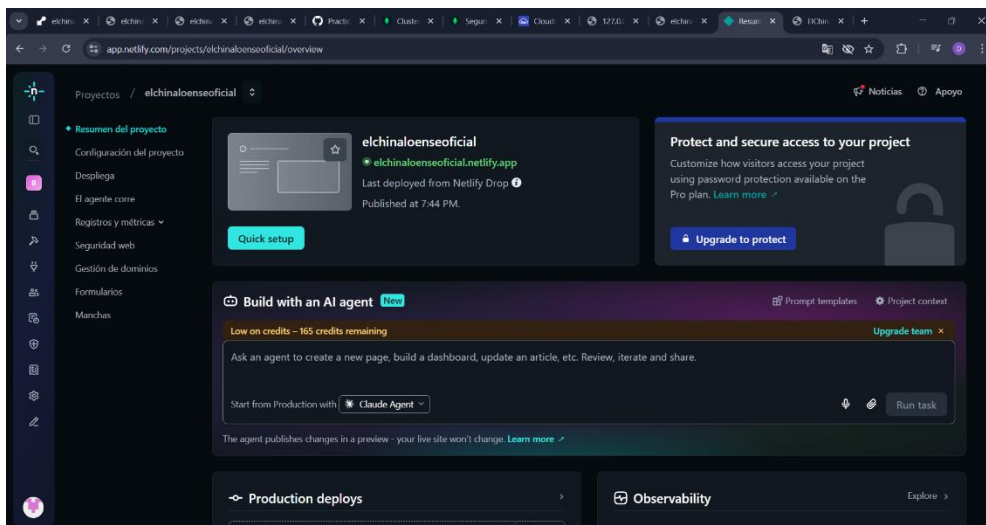
¿Para qué se utilizó?

- Publicar la aplicación Angular
- Generar una URL pública
- Permitir acceso desde cualquier dispositivo

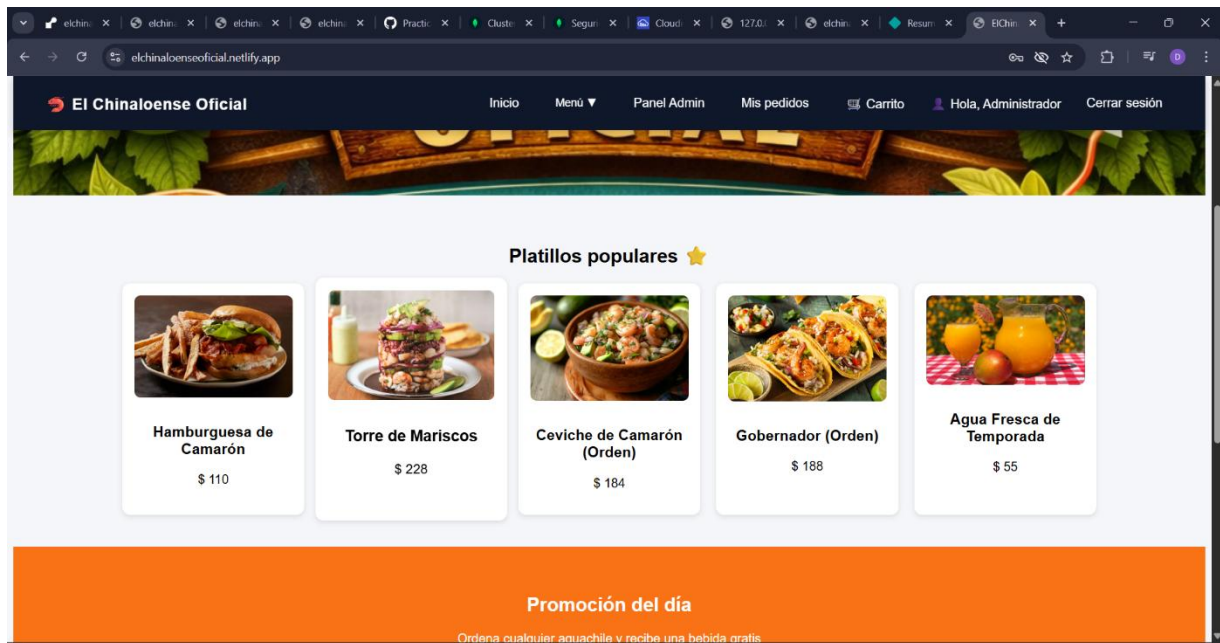
¿Por qué es importante?

Permite que los usuarios puedan interactuar con el sistema sin necesidad de instalar nada.

- **Página publicada:**



- URL funcionando:



MongoDB Atlas

MongoDB Atlas es la base de datos utilizada en la nube.

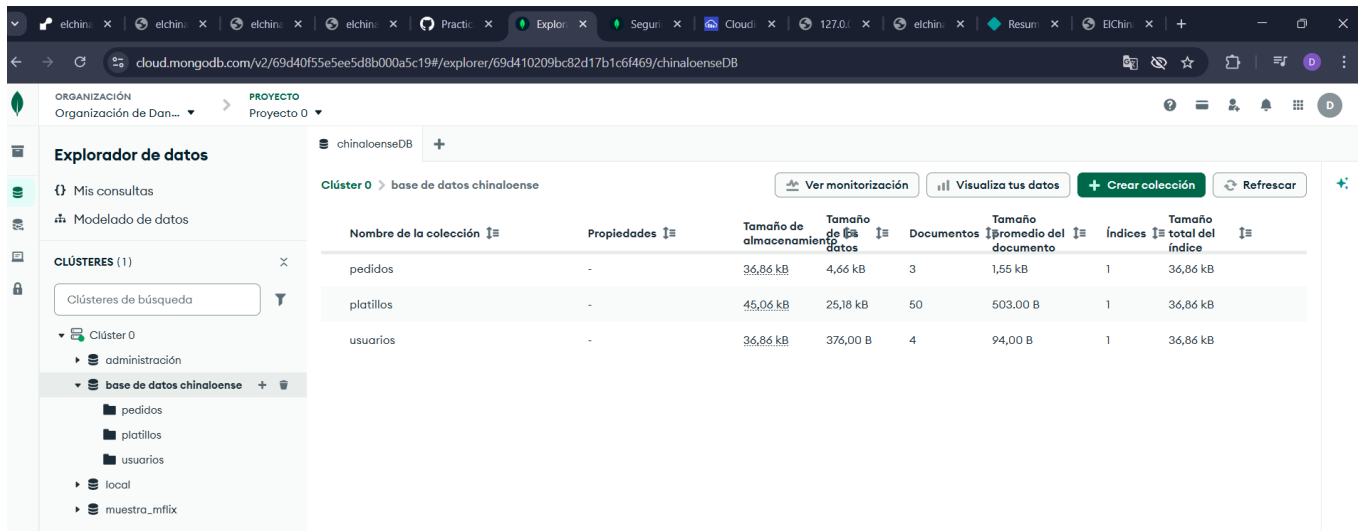
¿Para qué se utilizó?

- Almacenar usuarios
- Guardar pedidos
- Registrar productos

¿Por qué es importante?

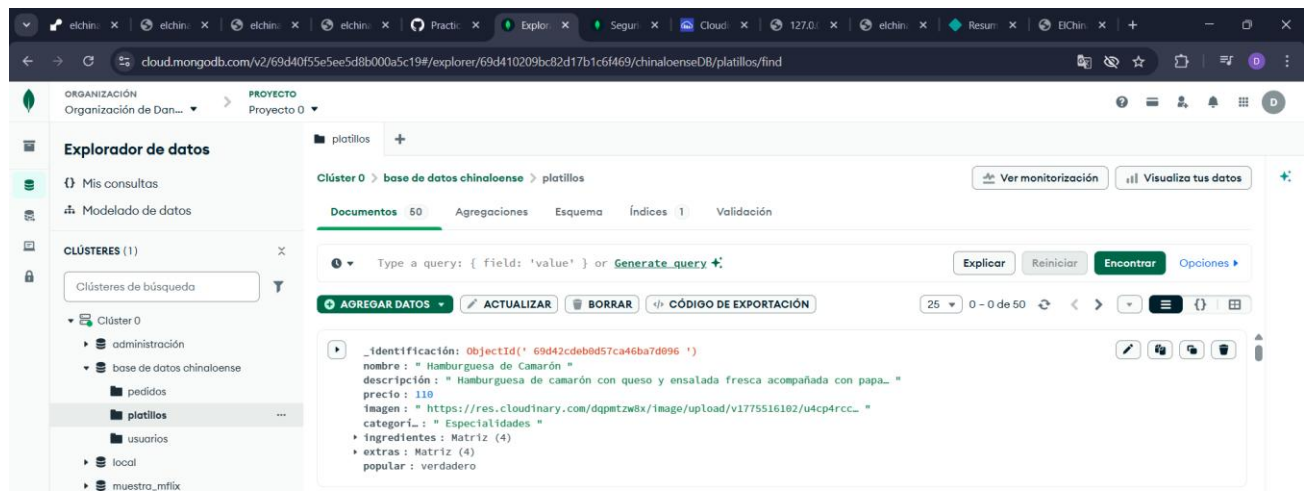
Permite guardar la información de forma permanente y acceder a ella en cualquier momento.

• Colecciones en MongoDB:

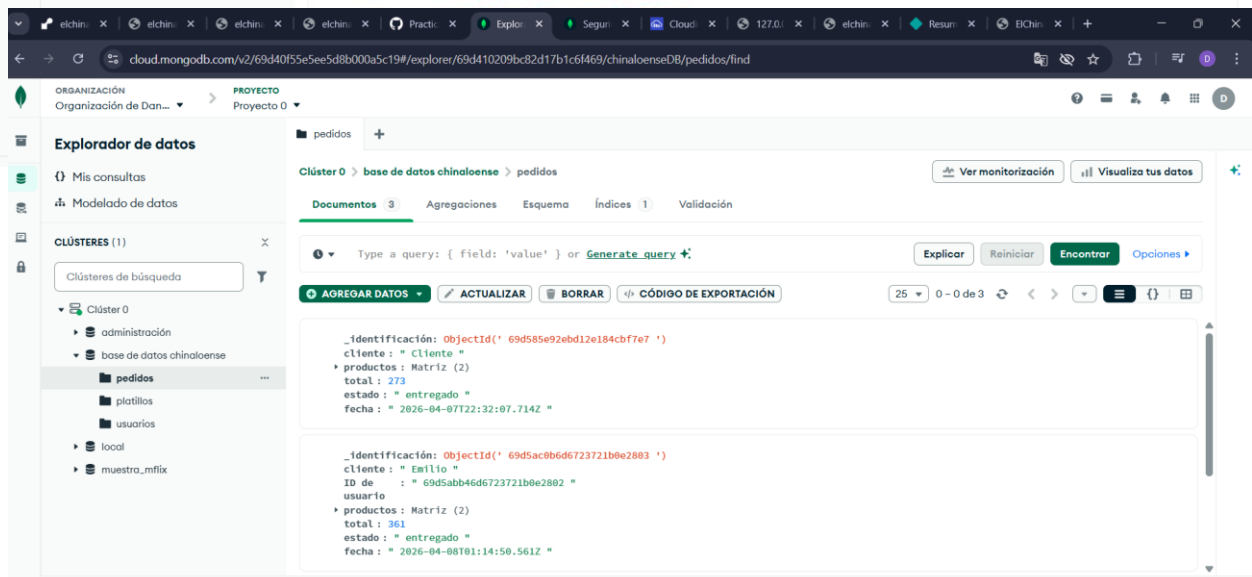


Nombre de la colección	Propiedades	Tamaño de almacenamiento	Tamaño de los datos	Documentos	Tamaño promedio del documento	Índices	Tamaño total del índice
pedidos	-	36,86 kB	4,66 kB	3	1,55 kB	1	36,86 kB
platillos	-	45,06 kB	25,18 kB	50	503,00 B	1	36,86 kB
usuarios	-	36,86 kB	376,00 B	4	94,00 B	1	36,86 kB

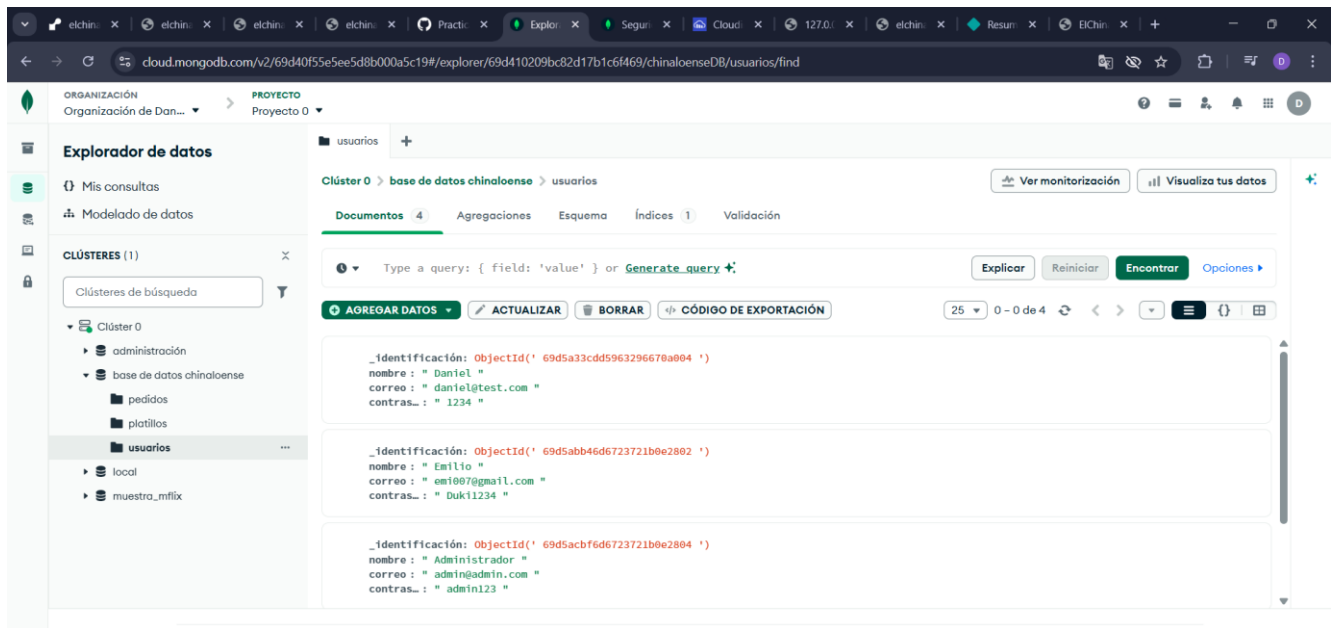
• Documentos guardados:



```
{ "_id": "69d43cde8d57ca46ba7d096", "nombre": "Hamburguesa de Camarón", "descripción": "Hamburguesa de camarón con queso y ensalada fresca acompañada con papa.", "precio": 110, "imagen": "https://res.cloudinary.com/dqpmz8x/image/upload/v1775516102/u4cp4rcc...", "categoría": "Especialidades", "ingredientes": "Matriz (4)", "extras": "Matriz (4)", "popular": "verdadero" }
```



```
{ "_id": "69d585e92ebd12e184cbf7e7", "cliente": "Cliente", "productos": "Matriz (2)", "total": 273, "estado": "entregado", "fecha": "2026-04-07T22:32:07.714Z" }, { "_id": "69d5ac8b6d6723721b0e2803", "cliente": "Emilio", "id": "69d5abb46d6723721b0e2802", "usuario": "usuario", "productos": "Matriz (2)", "total": 361, "estado": "entregado", "fecha": "2026-04-08T01:14:50.561Z" }
```



Thunder Client

Herramienta para probar el backend.

¿Para qué se utilizó?

- Enviar peticiones HTTP
- Ver respuestas del servidor
- Detectar errores

¿Por qué es importante?

Antes de conectar el frontend, se verificó que el backend funcionara correctamente.

4. SISTEMA DE AUTENTICACIÓN

El sistema de autenticación permite controlar el acceso de los usuarios y garantizar que cada uno tenga su propia sesión.

Registro de usuarios

Funcionamiento completo:

1. El usuario ingresa sus datos en el formulario
2. Angular captura estos datos
3. Se envían al backend mediante una petición POST
4. Flask valida si el usuario ya existe en MongoDB
5. Si no existe, se guarda en la base de datos

Importancia:

- Evita duplicidad de usuarios
 - Permite identificar a cada cliente
 - Es la base del sistema de acceso
-

Inicio de sesión (Login)

Funcionamiento:

1. El usuario ingresa sus credenciales
2. Angular envía los datos al backend
3. Flask busca coincidencias en MongoDB
4. Si coinciden, permite acceso

Importancia:

Permite que solo usuarios registrados puedan utilizar el sistema.

Manejo de sesión

Se utiliza **localStorage** para almacenar los datos del usuario.

¿Por qué?

- Mantiene la sesión activa
- Mejora la experiencia del usuario

5. CARRITO DE COMPRAS (EXPLICACIÓN COMPLETA)

El carrito permite al usuario seleccionar productos antes de realizar un pedido.

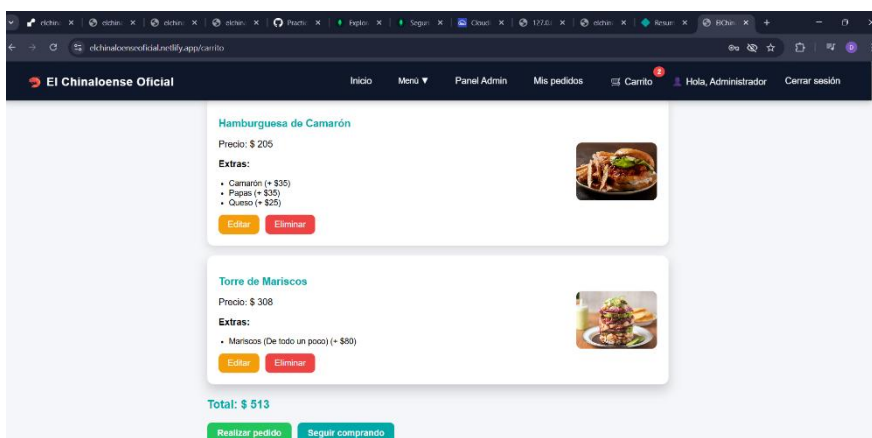
Funcionamiento detallado:

- Cada producto seleccionado se guarda en localStorage
- Se puede modificar cantidad
- Se puede eliminar productos
- Se calcula automáticamente el total

¿Por qué es importante?

- Permite revisar el pedido antes de enviarlo
- Mejora la experiencia del usuario
- Simula el comportamiento de sistemas reales de compra

- **Carrito con productos y total:**



6. SISTEMA DE PEDIDOS

Es la funcionalidad principal del sistema.

Funcionamiento paso a paso:

1. El usuario confirma su pedido
2. Angular envía los datos al backend
3. Flask recibe la información
4. Se guarda en MongoDB
5. El administrador puede visualizarlo

Importancia:

- Registra ventas
 - Organiza pedidos
 - Permite seguimiento
-

7. PANEL ADMINISTRATIVO

El panel administrativo permite gestionar todo el sistema.

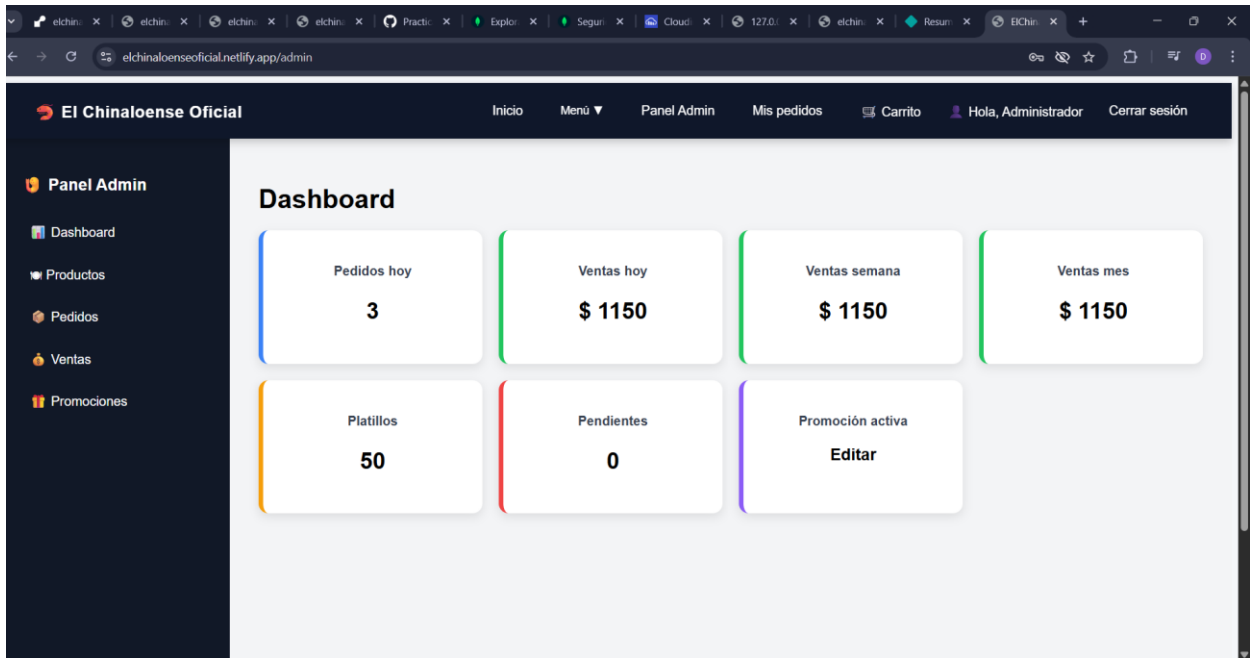
Funciones:

- Ver pedidos
- Cambiar estado
- Administrar productos
- Visualizar ventas

Importancia:

Permite al administrador tener control total del negocio.

• Dashboard:



8. INTEGRACIÓN FRONTEND - BACKEND

La integración entre el frontend y el backend es uno de los elementos más importantes del sistema, ya que permite la comunicación entre la interfaz de usuario (Angular) y la lógica del servidor (Flask).

Sin esta integración, el sistema no podría funcionar, ya que el frontend por sí solo solo muestra información, pero no puede procesarla ni almacenarla.

¿Cómo se realiza esta integración?

La comunicación se realiza mediante el protocolo HTTP, utilizando principalmente los métodos:

- **GET:** para obtener información (ej. productos, pedidos)
- **POST:** para enviar información (ej. registro, pedidos)
- **PUT:** para actualizar datos (ej. estado de pedidos)

Funcionamiento paso a paso

1. El usuario realiza una acción en la interfaz (ej. iniciar sesión o hacer un pedido)
 2. Angular utiliza el servicio **HttpClient** para enviar una petición HTTP al backend
 3. La petición incluye datos en formato JSON
 4. Flask recibe la petición en un endpoint específico
 5. Se procesa la información según la lógica definida
 6. Se interactúa con MongoDB si es necesario
 7. Flask genera una respuesta (éxito o error)
 8. Angular recibe la respuesta y actualiza la interfaz
-

Ejemplo real dentro del sistema

Cuando un usuario realiza un pedido:

- Angular envía un POST con los datos del carrito
 - Flask recibe el pedido
 - Se valida la información
 - Se guarda en MongoDB
 - Se devuelve una respuesta de confirmación
 - Angular muestra un mensaje al usuario
-

Importancia de esta integración

- Permite el funcionamiento dinámico del sistema
 - Evita recargar la página constantemente
 - Mejora la experiencia del usuario
 - Permite separar responsabilidades (frontend/backend)
-

9. DESPLIEGUE DEL BACKEND

El despliegue del backend consiste en publicar el servidor en internet para que pueda ser accedido desde cualquier lugar.

En este proyecto, se utilizó la plataforma **Render**.

¿Qué significa desplegar el backend?

Antes del despliegue, el backend solo funcionaba en la computadora local del desarrollador. Esto significa que nadie más podía acceder al sistema.

Al desplegarlo:

- El servidor se ejecuta en la nube
 - Está disponible las 24 horas
 - Puede recibir peticiones desde cualquier dispositivo conectado a internet
-

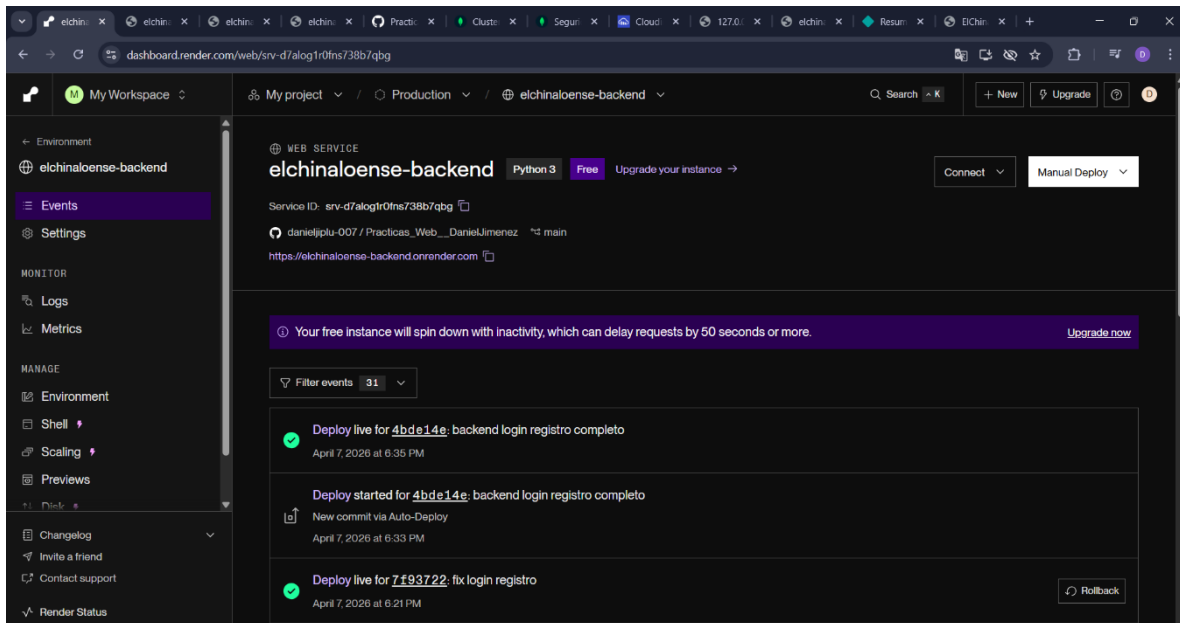
Proceso realizado

1. Se preparó el proyecto backend (Flask)
 2. Se subió el código a un repositorio
 3. Se configuró el servicio en Render
 4. Se definieron variables de entorno
 5. Se ejecutó el servidor en la nube
-

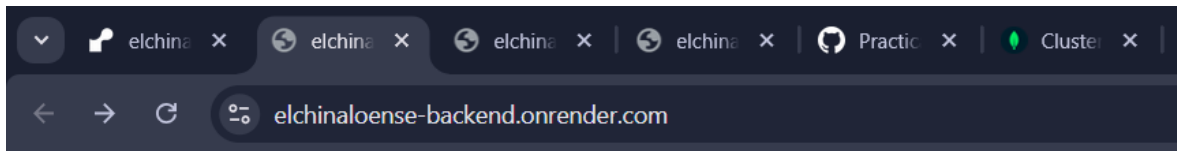
Importancia

- Permite que el frontend funcione correctamente en línea
 - Hace posible el uso real del sistema
 - Garantiza disponibilidad constante
-

- **Dashboard de Render:**



- **Backend activo:**



Backend funcionando

10. DESPLIEGUE DEL FRONTEND

El despliegue del frontend permite que la aplicación web sea accesible para cualquier usuario mediante un navegador.

Se utilizó la plataforma **Netlify**.

¿Qué implica este proceso?

- Convertir el proyecto Angular en archivos estáticos (HTML, CSS, JS)
- Subir estos archivos a un servidor
- Generar una URL pública

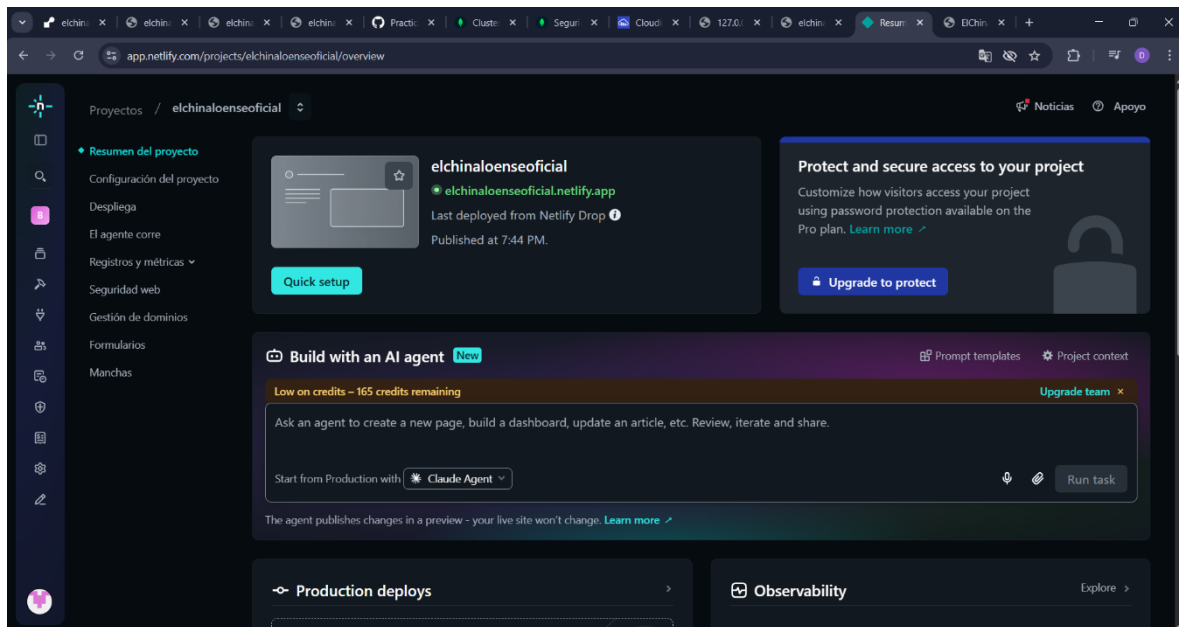
Proceso realizado

1. Se compiló el proyecto con ng build
 2. Se generó la carpeta de producción
 3. Se subieron los archivos a Netlify
 4. Se configuró la conexión con el backend
-

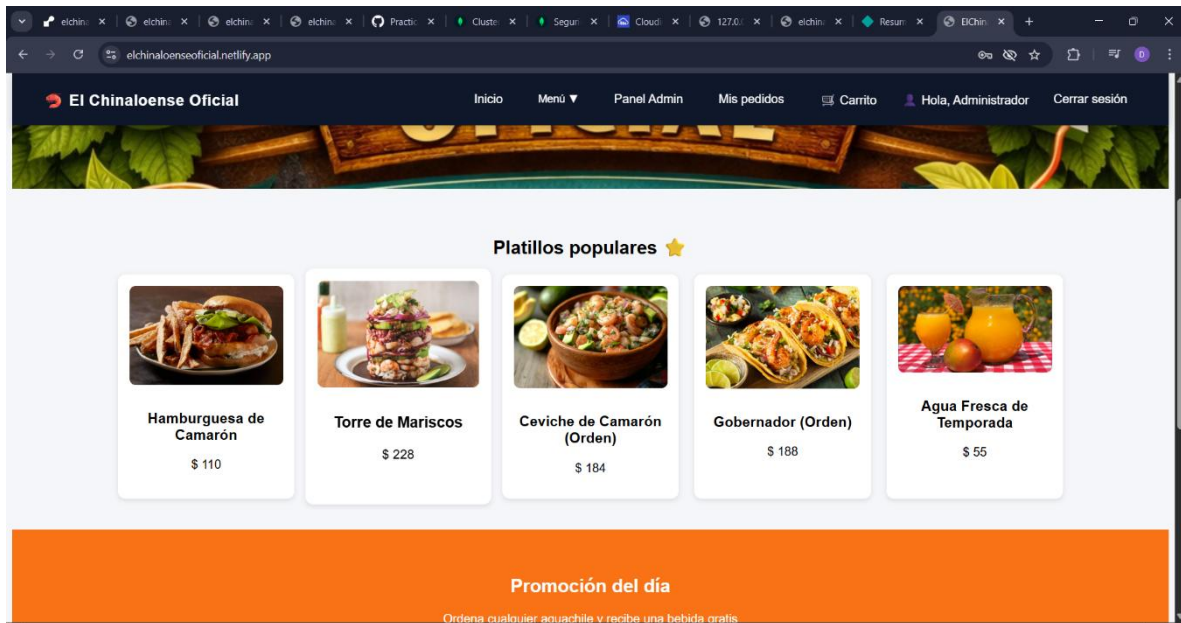
Importancia

- Permite acceder al sistema desde cualquier dispositivo
 - No requiere instalación
 - Mejora la accesibilidad del sistema
-

- **Página en línea:**



- **URL funcionando:**



II. PROBLEMAS Y SOLUCIONES

Durante el desarrollo del sistema se presentaron diversos errores, los cuales fueron fundamentales para el aprendizaje y mejora del proyecto.

✖ Error 404 (Not Found)

Problema:

El sistema no encontraba ciertas rutas del backend.

Causa:

Endpoints mal definidos o rutas incorrectas.

Solución:

Se revisaron y corrigieron las rutas en Flask, asegurando que coincidieran con las llamadas del frontend.

✖ Problemas con Observables en Angular

Problema:

Los datos no se mostraban en la interfaz.

Causa:

No se estaba utilizando correctamente la suscripción a los observables.

Solución:

Se implementó el uso de `.subscribe()` para obtener la respuesta de las peticiones HTTP.

✖ Manejo incorrecto de roles

Problema:

El sistema no distinguía correctamente entre tipos de usuario.

Causa:

El rol no se estaba guardando correctamente en la base de datos.

Solución:

Se modificó la lógica del backend para almacenar correctamente el rol del usuario.

✖ Error en el despliegue

Problema:

El backend no funcionaba correctamente en Render.

Causa:

Configuraciones incorrectas en variables de entorno o rutas.

Solución:

Se ajustaron configuraciones y dependencias necesarias para el correcto funcionamiento.

Importancia de esta sección

Documentar errores demuestra:

- Capacidad de análisis

- Resolución de problemas
 - Proceso de aprendizaje real
-

12. RESULTADOS Y EVIDENCIAS DEL DESPLIEGUE

El sistema desarrollado logró cumplir con los objetivos planteados desde el inicio del proyecto.

Resultados obtenidos

- Sistema de registro funcional
 - Inicio de sesión operativo
 - Carrito de compras dinámico
 - Sistema de pedidos completamente funcional
 - Panel administrativo activo
 - Aplicación desplegada en internet
-

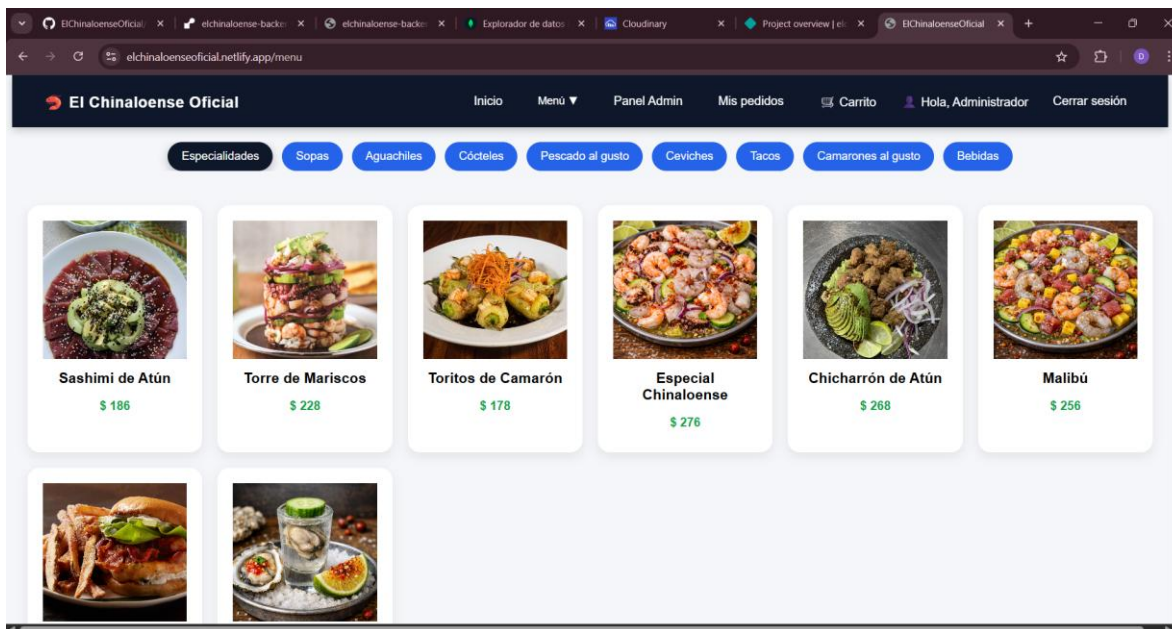
Impacto del sistema

- Reducción de errores en pedidos
 - Mejora en la organización del restaurante
 - Mayor rapidez en la atención
 - Experiencia más profesional para el cliente
-

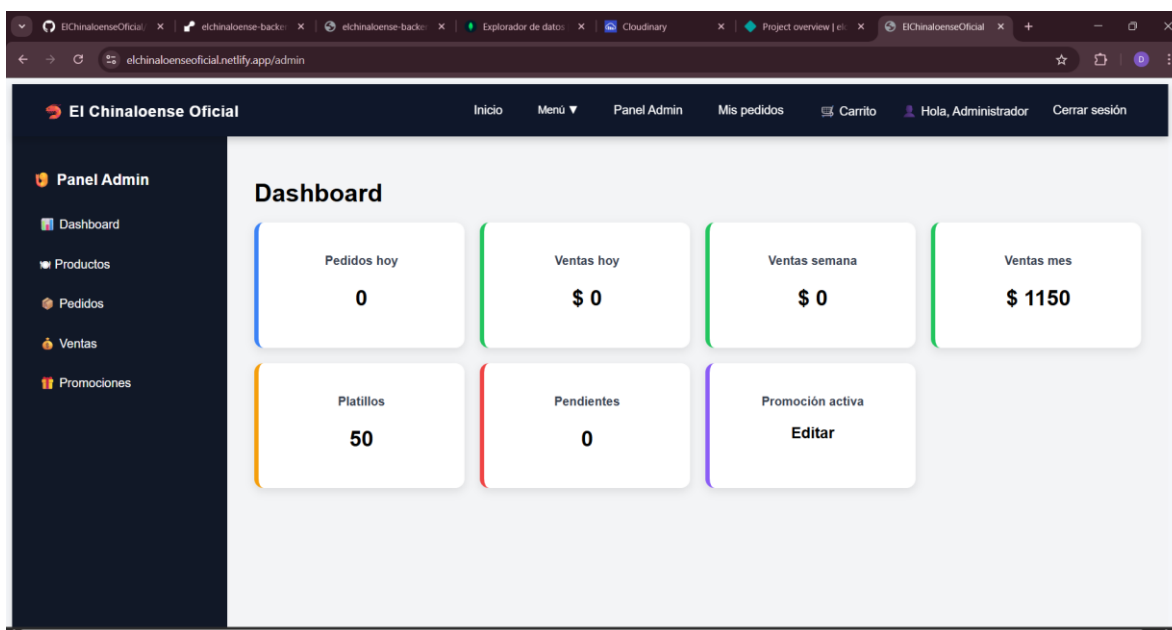
Inicio:



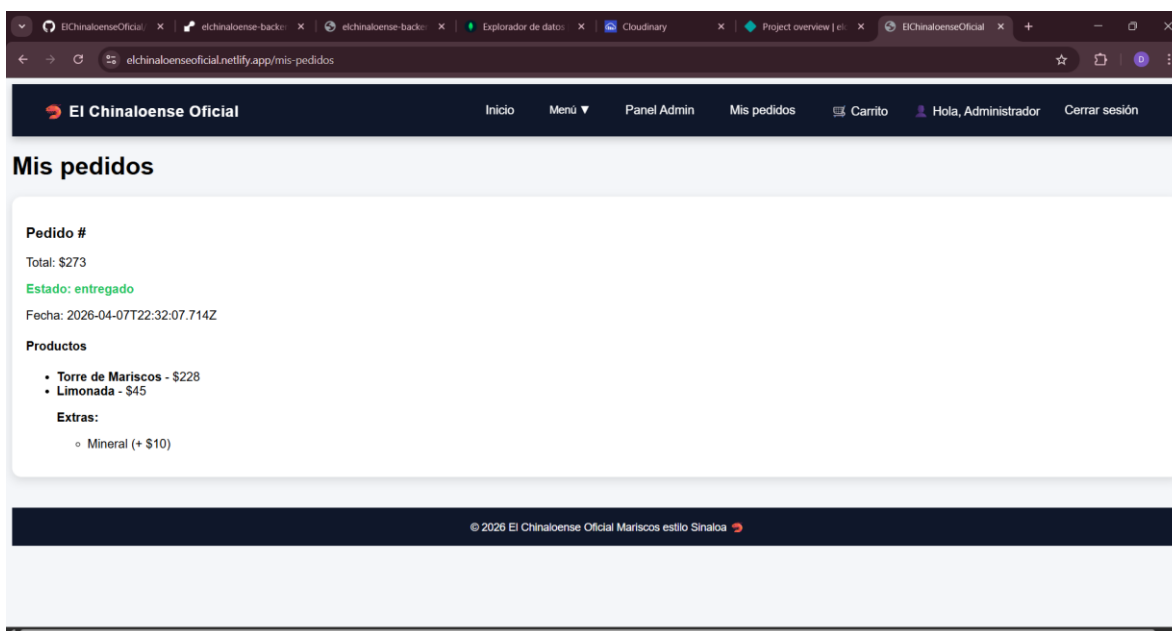
Menú:



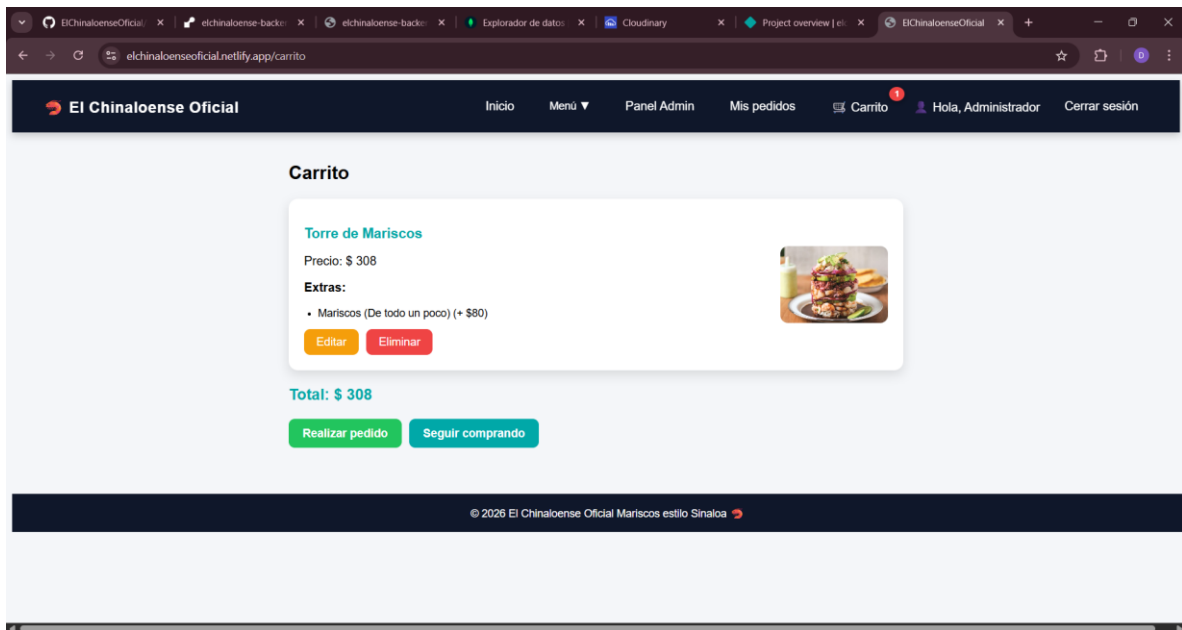
Panel admin:



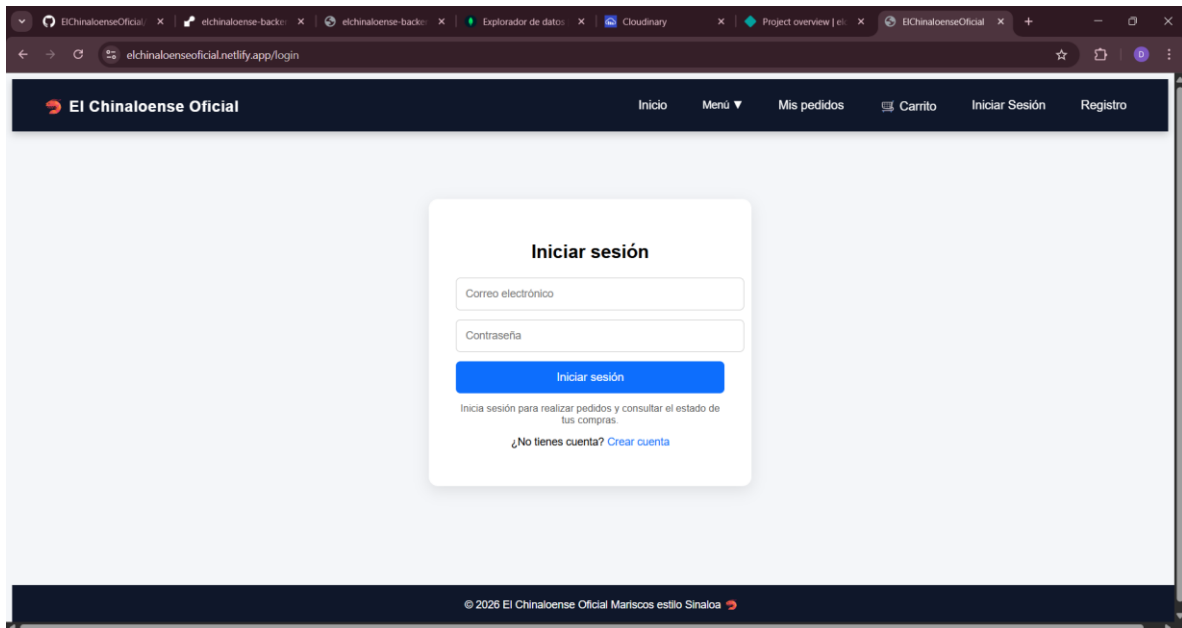
Pedidos:



Carrito:



Inicio de sesión:



Registro:

The screenshot shows a web browser with multiple tabs open, including 'ElChinaloenseOficial', 'elchinaloense back', 'Explorador de datos', 'Cloudinary', and 'Project overview | el'. The address bar shows 'elchinaloenseoficial.netlify.app/registro'. The website header features the 'El Chinaloense Oficial' logo and navigation links: 'Inicio', 'Menú', 'Mis pedidos', 'Carrito', 'Iniciar Sesión', and 'Registro'. The main content area displays a 'Crear cuenta' form with three input fields: 'Nombre', 'Correo electrónico', and 'Contraseña'. Below the fields is a blue 'Crear cuenta' button and a link that says '¿Ya tienes cuenta? Iniciar sesión'. The footer contains the copyright notice '© 2026 El Chinaloense Oficial Mariscos estilo Sinaloa'.

13. MEJORAS FUTURAS

Aunque el sistema es funcional, existen diversas mejoras que pueden implementarse para aumentar su calidad y escalabilidad.

Implementación de JWT

Permitiría mejorar la seguridad del sistema mediante autenticación basada en tokens.

Pagos en línea

Integrar pasarelas de pago para permitir compras directas desde la plataforma.

Notificaciones en tiempo real

Permitiría informar al usuario sobre el estado de su pedido.

Mejoras de seguridad

- Encriptación de contraseñas
 - Validaciones más estrictas
 - Protección contra ataques
-

Mejora de interfaz (UI/UX)

Hacer la aplicación más atractiva y fácil de usar.

14. CONCLUSIÓN

El desarrollo del sistema web “El Chinaloense Oficial” representa una solución tecnológica completa orientada a resolver una problemática real dentro del ámbito de la gestión de pedidos en un restaurante.

A lo largo del proyecto, se logró implementar una arquitectura cliente-servidor que permitió separar adecuadamente la interfaz de usuario y la lógica del sistema, facilitando el desarrollo, mantenimiento y escalabilidad de la aplicación.

El uso de tecnologías modernas como Angular en el frontend, Flask en el backend y MongoDB como base de datos, permitió construir un sistema robusto, dinámico y funcional, capaz de gestionar usuarios, pedidos y productos de manera eficiente.

Además, el proceso de despliegue en plataformas como Render y Netlify permitió llevar el sistema a un entorno real, donde puede ser utilizado desde cualquier dispositivo con acceso a internet, cumpliendo así con uno de los objetivos más importantes del proyecto: la accesibilidad.

Durante el desarrollo, se enfrentaron diversos retos técnicos, los cuales fueron superados mediante la investigación, prueba y corrección de errores, fortaleciendo así las habilidades en resolución de problemas y desarrollo de software.

En conclusión, este proyecto no solo cumple con los objetivos planteados, sino que también demuestra la capacidad de integrar múltiples tecnologías para desarrollar una solución completa, funcional y escalable, sentando las bases para futuras mejoras e implementación en un entorno real.

Link del repositorio para ver la documentación:

https://github.com/danieljplu-007/Practicas_Web__DanieJimenez